Questions: Introductory

1. Here are the screenshots provided of the ollama list showing 3 models installed

```
writing manifest
success
(base) faisalyusuf@Mac ~ % ollama list
NAME                ID              SIZE      MODIFIED
phi3:latest         4f2222927938    2.2 GB    17 seconds ago
mistral:latest      6577803aa9a0    4.4 GB    4 minutes ago
llama3.2:latest     a80c4f17acd5    2.0 GB    10 minutes ago
(base) faisalyusuf@Mac ~ %
```

```
Last login: Sat Feb 28 03:54:29 on ttys000
(base) faisalyusuf@Mac ~ % ollama serve
zsh: command not found: ollama
(base) faisalyusuf@Mac ~ % ollama serve
Error: listen tcp 127.0.0.1:11434: bind: address already in use
(base) faisalyusuf@Mac ~ % ollama pull llama3.2
pulling manifest
pulling dde5aa3fc5ff: 100%  ▕▕▕
```

2. Here is the command line and some responses by the LLM :

```
(base) faisalyusuf@Mac llm-log-analyzer-main % python3 -m venv venv
(base) faisalyusuf@Mac llm-log-analyzer-main % source venv/bin/activate
(venv) (base) faisalyusuf@Mac llm-log-analyzer-main % pip install -r requirements.txt
Collecting pyyaml (from -r requirements.txt (line 1))
  Downloading pyyaml-6.0.3-cp314-cp314-macosx_11_0_arm64.whl.metadata (2.4 kB)
Collecting requests (from -r requirements.txt (line 2))
  Downloading requests-2.33.0-py3-none-any.whl.metadata (5.1 kB)
Collecting python-dotenv (from -r requirements.txt (line 3))
  Downloading python_dotenv-1.2.2-py3-none-any.whl.metadata (27 kB)
Collecting openai (from -r requirements.txt (line 4))
  Downloading openai-2.30.0-py3-none-any.whl.metadata (29 kB)
Collecting charset_normalizer<4,>=2 (from requests->-r requirements.txt (line 2))
  Downloading charset_normalizer-3.4.6-cp314-cp314-macosx_10_15_universal2.whl.metadata (40 kB)
Collecting idna<4,>=2.5 (from requests->-r requirements.txt (line 2))
  Downloading idna-3.11-py3-none-any.whl.metadata (8.4 kB)
Collecting urllib3<3,>=1.26 (from requests->-r requirements.txt (line 2))
  Downloading urllib3-2.6.3-py3-none-any.whl.metadata (6.9 kB)
Collecting certifi>=2023.5.7 (from requests->-r requirements.txt (line 2))
  Downloading certifi-2026.2.25-py3-none-any.whl.metadata (2.5 kB)
```

```
(venv) (base) faisalyusuf@Mac llm-log-analyzer-main % python log-analyzer.py -f test-auth.log -c prompt.yaml

(venv) (base) faisalyusuf@Mac llm-log-analyzer-main % python log-analyzer.py -f test-auth.log -c prompt.yaml -n 16
Starting the LLM log analyzer
-> Sending POST request to Ollama

============================================================
LLM RESPONSE:
============================================================
Abnormal or malicious behavior observed:

1. Connection closure by an unknown client (101.43.42.253) without a valid reason.
2. Multiple consecutive connection closures and reopenings for the same IP address from different ports, indicating po
tential brute-force attempts.
3. Repeated warnings about deprecated configuration options (RSAAuthentication and RhostsRSAAuthentication), which may
 indicate an outdated or insecure SSH setup.
4. A successful login attempt for an unknown user ('project') without a valid password, potentially due to a configura
tion issue or a malicious exploit.
5. Connection closures by the same IP address from different ports, with no apparent reason for these connections.
6. An authentication failure for the 'root' user without a valid password, followed by a successful connection closure
```

<u>My Interpretation based on the responses:</u> The logs show some suspicious usage, such as repeated connections and closures by the same IP, which may be an indication of brute-force usage. Authentication failures are also present, such as an attempt to access the root account, which is often a target of attack. An event such as a successful login of a stranger could be an indication of an unconfigured system or a security violation. There were also old SSH configurations that were identified, it could compromise the security of the systems. On the whole, the system demonstrates the indications of a possible intrusion and a configuration weakness.

Log Preparition:

1. Writing a python script :

!)
```
import pandas as pd

INPUT_CSV = "OpenSSH_2k.log_structured.csv"
OUTPUT_LOG = "openssh_converted.log"
```

```python
def build_syslog_line(row):
    return f"{row['Date']} {row['Day']} {row['Time']} {row['Component']} sshd[{row['Pid']}]:
{row['Content']}"

def main():
    df = pd.read_csv(INPUT_CSV)

    df["syslog"] = df.apply(build_syslog_line, axis=1)
    df["syslog"].to_csv(OUTPUT_LOG, index=False, header=False)

    print("=== Conversion Summary ===")
    print(f"Total lines converted: {len(df)}")
    print(f"Unique event types: {df['EventId'].nunique()}")
    print(f"Most common event ID: {df['EventId'].value_counts().idxmax()}")
    print()

    print("=== First 5 lines of openssh_converted.log ===")
    for line in df["syslog"].head(5):
        print(line)
    print()

    invalid_users = df[df["Content"].str.contains("Invalid user", na=False)].copy()
    invalid_users["ip"] = invalid_users["Content"].str.extract(r"from ([0-9]+\.[0-9]+\.[0-9]+\.[0-9]+)")
    invalid_users["username"] = invalid_users["Content"].str.extract(r"Invalid user (\S+) from")

    print("=== Manual Inspection ===")
    print(f'Entries containing "Invalid user": {len(invalid_users)}')
    print(f'Unique source IP addresses: {invalid_users["ip"].nunique()}')
    print(f'Most frequently targeted invalid username: {invalid_users["username"].value_counts().idxmax()}')

if __name__ == "__main__":
    main()
```

!!) screenshot of terminal output :

```
(venv) (base) faisalyusuf@Mac llm-log-analyzer-main % python convert_openssh_logs.py
=== Conversion Summary ===
Total lines converted: 2000
Unique event types: 27
Most common event ID: E24

=== First 5 lines of openssh_converted.log ===
Dec 10 06:55:46 LabSZ sshd[24200]: reverse mapping checking getaddrinfo for ns.marryaldkfaczcz.com [173.234.31.186] fa
iled - POSSIBLE BREAK-IN ATTEMPT!
Dec 10 06:55:46 LabSZ sshd[24200]: Invalid user webmaster from 173.234.31.186
Dec 10 06:55:46 LabSZ sshd[24200]: input_userauth_request: invalid user webmaster [preauth]
Dec 10 06:55:46 LabSZ sshd[24200]: pam_unix(sshd:auth): check pass; user unknown
Dec 10 06:55:46 LabSZ sshd[24200]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= r
host=173.234.31.186

=== Manual Inspection ===
Entries containing "Invalid user": 113
Unique source IP addresses: 19
Most frequently targeted invalid username: admin
(venv) (base) faisalyusuf@Mac llm-log-analyzer-main %
```

!!!) The first 5 lines from openssh_converted.log

Dec 10 06:55:46 LabSZ sshd[24200]: reverse mapping checking getaddrinfo for ns.marryaldkfaczcz.com [173.234.31.186] failed - POSSIBLE BREAK-IN ATTEMPT!
Dec 10 06:55:46 LabSZ sshd[24200]: Invalid user webmaster from 173.234.31.186
Dec 10 06:55:46 LabSZ sshd[24200]: input_userauth_request: invalid user webmaster [preauth]
Dec 10 06:55:46 LabSZ sshd[24200]: pam_unix(sshd:auth): check pass; user unknown
Dec 10 06:55:46 LabSZ sshd[24200]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=173.234.31.186

2. Before moving to AI Analysis, these are the questions answered from the output:
   a) 113 entries containing 'invalid user.'
   b) 19 unique source IP addresses
   c) Admin is the most frequently targeted invalid username

I have already attached my code and output above.


```
(venv) (base) faisalyusuf@Mac llm-log-analyzer-main % python convert_openssh_logs.py
=== Conversion Summary ===
Total lines converted: 2000
Unique event types: 27
Most common event ID: E24

=== First 5 lines of openssh_converted.log ===
Dec 10 06:55:46 LabSZ sshd[24200]: reverse mapping checking getaddrinfo for ns.marryaldkfaczcz.com [173.234.31.186] fa
iled - POSSIBLE BREAK-IN ATTEMPT!
Dec 10 06:55:46 LabSZ sshd[24200]: Invalid user webmaster from 173.234.31.186
Dec 10 06:55:46 LabSZ sshd[24200]: input_userauth_request: invalid user webmaster [preauth]
Dec 10 06:55:46 LabSZ sshd[24200]: pam_unix(sshd:auth): check pass; user unknown
Dec 10 06:55:46 LabSZ sshd[24200]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= r
host=173.234.31.186

=== Manual Inspection ===
Entries containing "Invalid user": 113
Unique source IP addresses: 19
Most frequently targeted invalid username: admin
(venv) (base) faisalyusuf@Mac llm-log-analyzer-main %
```


1. **Brute Force Detection Prompting:** My edited file is : prompt: |

You are a SSH intrusion analyst. Test the OpenSSH authentication logs given and only analyse them in terms of brute-force behaviour.

Important scope rules:
- Only concentrate on brute-force and password-guessing behavior.
- Do NOT give remediation, mitigation or hardening guidance.
- Analysis only.

Processing instructions:
- First, make an effort to treat the entire input given as a single dataset.
- When the input is too big, incomplete or results in loss of detail, explicitly state that analysis in full context can be unreliable and proceed analysis in sequential chunks maintaining findings across chunks.
- Consider the repetitive activities that have the same IP within the dataset as cumulative, although it might be observed in the different sections.
- Where feasible, can be used to deduce attacker behavior based on recurring patterns of SSHs like:
  - "Failed password"
  - "Invalid user"
  - attempts by the same IP sources are repeated.
  - recurrent attack on username of the same ip.
  - rapid retries within the course of a brief time.

Required tasks:
1. Determine the IP addresses of sources where the failed logins have been occurring.
2. Distinguish between:
   - passwords failures on existing or attempted accounts.
   - invalid username attempts which indicate username enumeration.
3. Name the IPs that are most likely to be performing brute-force.
4. Give a severity rating to each of the identified IP: Low, Medium, High, and Critical.
5. List any usernames that were systematically attacked, where showing up.
6. Record the behavior being more similar to password spraying, dictionary attack, or username enumeration.

Output format:
Give your response using the following format:

Brute-Force Summary:
- 2-4 sentence summary of the presence/absence of the brute-force behavior.

Potential Brute-Force Attackers:
In the case of every suspicious IP, provide:
- IP address:
- Observed behavior:
- Failed password attempts:
- Invalid username attempts:
- Targeted usernames:
- Classification of attack patterns:
- Severity:

Analytical Notes:
- Have a clear differentiation between probing and guessing of passwords by invalid users.
- State whether confidence is truncated, chunk-sized or out of context.

Here the command :

```
Most frequently targeted invalid username: admin
(venv) (base) faisalyusuf@Mac llm-log-analyzer-main % python log-analyzer.py -f openssh_converted.log -c edit_brute_fo
rce.yaml -m llama3.2 -n 2000 -o brute_force_output.txt
```

In my report i have attached everything :
-> Sending POST request to Ollama

==========
LLM RESPONSE:
==========
This is a chronicle of unsuccessful attempts of SSH logging into a server with OpenSSH. The log displays the IP address of the requesting device, the user name being used to log in, and the cause of failure (which is usually one of the following:

* failed auth or password incorrect - wrong password.
* invalid user - unknown or invalid name of the user.
* no such user - user is not present in the system.
* `bad service name` - wrong service name.
* `bad host` - incorrect hostname
* unknown key - authentication of a private key by the user failed.
* no valid authentication methods available (database: no valid auth) - no authentication method (e.g. public key) is turned on.
* Connection with the remote device has been closed.

The following are some of the observations that can be made out of the log:

1. **Multiple attempts with the same IP address: This is several login attempts with the same IP address and this may be an indication of someone trying to brute-force or guess the password.
2. **Types of usernames and services tested: The log indicated that a number of usernames (e.g. root, user, guest) and services (e.g. ssh, ftp, sftp) were tested.
3. **There are no authentication schemes that allow password-based logins: Since most current systems turn off password-based logins by default and instead demand the use of public key or Kerberos authentication, the high number of failed logins with passwords is unexpected.
4. The frequency of attempts: There seems to be no restriction on the attempt frequency: there are no restrictions on the number of attempts by a single IP address that can be traced within a certain period, which can be a problem in case the system is opening to brute-force attacks.

To improve security:

1. **Allow or deny public key or Kerberos authentication: It is suggested to consider the use of public key or Kerberos authentication on all SSH connections.
2. **Apply rate control and blockout IP addresses: configure rate controls on attempts to log in with one IP address, and block IP addresses that attempt to do so more than the rate control allows to avoid brute force attacks.

3. **Turn the password-based logins off: In any possible way, turn off the possibility to use passwords as an authentication method.
4. **View system logs on a regular basis**: Periodically check on the logs of the system in order to identify any suspicious activity, including a series of unsuccessful attempts to log into it by the same IP address.

It is important to mention that this is a hypothetical analysis given the content and the form of the logs that are given. The real security guidelines can be different based on the particular system set up, network setup and so on.

```
(venv) (base) faisalyusuf@Mac llm-log-analyzer-main % python log-analyzer.py -f openssh_converted.log -c edit_brute_fo
rce.yaml -m llama3.2 -n 2000 -o brute_force_output.txt
Starting the LLM log analyzer
-> Sending POST request to Ollama

============================================================
LLM RESPONSE:
============================================================
This is a log of failed SSH login attempts to a server running OpenSSH. The log shows the IP address of the attempting
 device, the username attempted to log in with, and the reason for the failure (usually one of the following:

* `failed auth` or `password incorrect` - incorrect password
* `invalid user` - unknown or invalid username
* `no such user` - user does not exist on the system
* `bad service name` - incorrect service name
* `bad host` - incorrect hostname
* `unknown key` - private key authentication failed
* `no valid authentication methods available (database: no valid auth)` - authentication method (e.g. public key) is n
ot enabled
* `Received disconnect from ...: 11: Bye Bye` - connection closed by remote device

Here are some observations that can be made from the log:

1. **Multiple attempts from a single IP address**: There are multiple login attempts from a single IP address, suggest
ing that someone is trying to brute-force or guess passwords.
2. **Variety of usernames and services attempted**: The log shows a range of usernames (e.g., `root`, `user`, `guest`)
 being tried, as well as different services (e.g., `ssh`, `ftp`, `sftp`).
3. **No authentication methods enabled for password-based logins**: Given that many modern systems disable password-ba
sed logins by default and instead require public key or Kerberos authentication, it's surprising to see a high number
of failed login attempts using passwords.
4. **No apparent limit on attempt frequency**: The log does not show any limits being enforced on the number of attemp
ts from a single IP address within a certain timeframe, which could be an issue if the system is exposed to brute-forc
e attacks.

To improve security:

1. **Enable public key or Kerberos authentication**: Consider requiring public key or Kerberos authentication for all
SSH connections.
2. **Implement rate limiting and IP blocking**: Set up rate limits on login attempts from a single IP address, and blo
ck IP addresses that exceed these limits to prevent brute-force attacks.
3. **Disable password-based logins**: Remove the ability to use passwords as an authentication method whenever possibl
e.
4. **Monitor system logs regularly**: Regularly review system logs to detect unusual activity, such as multiple failed
 login attempts from a single IP address.

Please note that this is just a hypothetical analysis based on the provided log format and content. The actual securit
y recommendations may vary depending on the specific system configuration, network environment, and other factors.
============================================================

(venv) (base) faisalyusuf@Mac llm-log-analyzer-main %
```

My evaluation is : The output only proved useful to an extent since it determined that the content of the log was repeated failed attempts of SSH logins and probable brute force activity, but it was too generic. It failed to explicitly provide the list of names of specific IP addresses that were involved and failed to appropriately differentiate between unsuccessful username and unsuccessful passwords in an organized manner. The other weakness is that it involved remediation recommendations although the prompt specifically asked that only an analysis be done. This is an indication that the model was not entirely observant of the directions on the YAML

prompt. In general, the answer was a general reflection of the issue of security at large, but this answer was not the part of a brute-force analysis that requires specificity and discipline.

I have checked with 500 lines and then for smaller chunks :
venv) (base) faisalyusuf@Mac llm-log-analyzer-main % python log-analyzer.py -f openssh_converted.log -c edit_brute_force.yaml -m llama3.2 -n 500 -o brute_force_output_500.txt

```
python log-analyzer.py -f openssh_converted.log -c edit_brute_force.yaml -m mistral -n 500 -o brute_force_mistral.tx
```

2. Anomaly & Timing Detection

I have edited the anomaly detection. My edit file is :

starter_anomaly.yaml

```
prompt: |
  You are a security analyst specializing in SSH authentication logs and temporal anomaly
detection.

  Your task is to analyze the provided OpenSSH log entries for timing-based anomalies and
unusual event sequences.

  Scope:
  - Focus on anomaly and timing detection only.
  - Do NOT provide remediation advice.
  - Analysis only.

  Processing instructions:
  - First, try to analyze the full provided dataset as one continuous timeline.
  - If the input is too large, too vague, or causes loss of detail, explicitly say that full-
context analysis may be unreliable and continue by analyzing the logs in sequential chunks whil
preserving findings across chunks.
  - Treat timestamps as important. Pay close attention to bursts of events within short periods
  - Track repeated attempts from the same IP across the data when possible.

  Required tasks:
  1. Look for rapid repeated attempts within short time windows.
  2. Identify unusual sequences of events, such as:
     - repeated invalid user attempts
     - repeated failed passwords
     - immediate disconnects after authentication failures
     - sudden switches between usernames from the same IP
     - bursts of activity from one or more IPs in a very short interval
  3. Flag any events or sequences that stand out as anomalous compared to the rest of the log.
  4. Report the specific timestamps or timestamp ranges where anomalous clusters occur.
```

Here are my given prompts:

```
(venv) (base) faisalyusuf@Mac llm-log-analyzer-main % python log-analyzer.py -f openssh_converted.log -c starter_anoma
ly.yaml -m llama3.2 -n 2000 -o anomaly_full.txt
python log-analyzer.py -f openssh_converted.log -c starter_anomaly.yaml -m llama3.2 -n 500 -o anomaly_500.txt
Starting the LLM log analyzer
-> Sending POST request to Ollama
```

Based on this :

```
============================================================
LLM RESPONSE:
============================================================
This is a log file of failed SSH login attempts on a Linux server. Here's a breakdown of the most common attacks and t
heir corresponding error messages:

1. **Invalid username or password**:
        * "invalid_user" (e.g., `test`, `guest`): These users are not defined in the system's user database, causing a
uthentication to fail.
2. **No authentication methods available**:
        * `auth failure`: The server has configured its SSH daemon to only allow specific authentication methods, such
 as key-based or password authentication, but none of the provided credentials match any of these methods.
3. **Authentication failed due to incorrect password**:
        * (empty password): This suggests that the user attempted to login with an empty password, which is not allowe
d.

Some patterns emerge from this log:

* Many attempts come from a single IP address (`103.99.0.122`), indicating a potential brute-force attack.
* Some usernames are repeated multiple times (e.g., `test`, `guest`, `user`), suggesting that an attacker is trying di
fferent usernames to crack the password or find a valid one.
* Some passwords appear to be attempts to crack common passwords, such as single-digit numbers (`1`, `2`, etc.).

To improve security:

* Consider increasing the difficulty of brute-force attacks by implementing rate limiting and IP blocking on login att
empts.
* Use strong authentication methods like public key authentication or multi-factor authentication whenever possible.
* Regularly review system logs to detect suspicious activity and adjust security configurations accordingly.

If you need further assistance, please provide more information about your specific situation, such as:

* The operating system running the SSH server
* Any existing security measures in place (e.g., password policies, IP blocking)
* The desired level of security and any specific requirements
============================================================
```

```
============================================================
LLM RESPONSE:
============================================================
The SSH login attempts logged in the provided output indicate a variety of usernames and their corresponding authentic
ation outcomes.

Here's a breakdown of some of the notable failed login attempts:

1.  **cisco** (Attempt #21):
    *   The user "cisco" was attempting to log in with the default password.
    *   Since the "check pass" command fails when trying to authenticate an unknown username with an empty password, t
his suggests that no authentication attempt was made for a valid username.
2.  **test** (Attempt #34):
    *   Similar to "cisco", another failed login attempt was for a user named "test".
3.  **user** (Attempt #42):
    *   An attempt was also made by a user trying to log in with the username "user".

These are just a few examples of the many failed authentication attempts logged during this period.

Here is a summary of these attempted usernames along with their outcomes:

| Username | Outcome |
| --- | --- |
| cisco | Failure |
| test | Failure |
| user | Failure |

It seems like there's an effort to brute-force common and potentially default SSH login credentials, but none have bee
n successful so far.
============================================================
```

My evaluation is: The anomaly prompt proved to be more efficient in the cases when it directly requested the model to concentrate on the timing windows, event bursts and unusual event order instead of providing a broad security overview. In the 2000-line run, the response was more extensive, and occasionally missed precise timestamp clusters, yet the 500-line run enhanced the potential of the model to bring out quick repeated efforts and suspicious groupings of occasions. The best results were the ones that found short bursts of authentication failure and follow-up events, e.g., repeated invalid-user attempts within a small time interval. Other flagged anomalies were real since they were a result of focused attack behavior as opposed to single failed logins. Nevertheless, the model was still prone to overgeneralization and in certain cases commonly occurring SSH noise was considered anomalous when the prompt was not stringent enough. In general, the most successful prompts issued the requests requiring the time-based clustering and presenting the anomalies in a structured way.

3. **Threat Actor Profiling Acting :**

edit_threat_actor_profiling.yaml — Edited

```
prompt: |
  You are a security analyst specializing in attacker profiling from SSH authentication logs.

  Your task is to profile suspicious actors based on source IP activity in the provided OpenSSH logs.

  Scope:
  - Focus only on attacker profiling and behavioral grouping by source IP.
  - Do NOT provide remediation advice.
  - Analysis only.

  Processing instructions:
  - First, attempt to analyze the full provided dataset as one complete body of evidence.
  - If the input appears too large or the analysis becomes vague, explicitly note that full-context profiling may be unreliable and continue
by analyzing the logs in sequential chunks while preserving cumulative findings across chunks.
  - Group activity by source IP address.
  - Treat repeated behavior from the same IP as part of one actor profile, even if it appears in different sections.

  Required tasks:
  1. Group suspicious activity by source IP address.
  2. For each attacker IP, describe:
     - what they attempted
     - how many times they appeared to try
     - what usernames they targeted
     - when the activity occurred
  3. Characterize each actor's attack style as one of the following when possible:
     - opportunistic scan
     - targeted attack
     - credential stuffing
     - username enumeration
     - brute-force password guessing
  4. Mention whether the actor appears noisy, persistent, broad, or focused.
  5. Provide a structured section for each suspicious IP.

  Output format:
  Threat Actor Summary:
  - Brief 2-4 sentence overview of the main attacker patterns in the log.

  Per-IP Profiles:
  For each suspicious IP, provide:
  - Source IP:
  - Approximate time(s) observed:
  - Observed actions:
  - Targeted usernames:
  - Estimated frequency or repetition:
  - Attack style:
  - Severity: Low / Medium / High / Critical
  - Confidence: Low / Medium / High

  Cross-Actor Comparison:
  - Identify which IPs appear most aggressive.
  - Identify which appear opportunistic versus more targeted.

  Final Constraint:
  - Do not include remediation or defensive recommendations.
```

My LLM response after giving the promt :

convert_openssh_logs.py | edit_brute_force.yaml | edit_threat_actor_profiling.yaml | OpenSSH_2k.log_structured.csv

edit_threat_actor_profiling.yaml

```yaml
prompt: |
  You are a security analyst specializing in attacker profiling from SSH authentication logs.

  Your task is to profile suspicious actors based on source IP activity in the provided OpenSSH logs.

  Scope:
  - Focus only on attacker profiling and behavioral grouping by source IP.
  - Do NOT provide remediation advice.
```

PROBLEMS 100 | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS

```
(venv) (base) faisalyusuf@Mac llm-log-analyzer-main % python log-analyzer.py -f openssh_converted.log -c edit_threat_actor_profiling.yaml -m llama3.2 -n 200
0 -o threat_actor_full.txt
============================================================
LLM RESPONSE:
============================================================
It appears that the logs are from a SSH server, and they show a series of failed login attempts from different IP addresses using various usernames.

The failures can be categorized into three groups:

1. **Unknown/Invalid Users**: There are several failed login attempts with unknown or invalid usernames, such as `cisco`, `guest`, `test`, `user`. These att
empts are likely from bots or malicious users trying to guess usernames.
2. **Incorrect Passwords**: Many of the failures are due to incorrect passwords entered by legitimate users. The logs show that some users tried multiple in
correct passwords before giving up.
3. **Successful Logins**: There are no successful login entries in the logs, suggesting that either the SSH server is configured correctly or the clients co
nnecting to it have not been successfully authenticated.

To improve security, I would recommend:

1. Implementing a rate limiting mechanism to prevent brute-force attacks on usernames and passwords.
2. Enabling IP address blocking for known malicious IP addresses.
3. Configuring the SSH server to use stronger password requirements, such as requiring uppercase letters, numbers, and special characters.
4. Regularly monitoring login attempts and implementing alerts for suspicious activity.

Additionally, it would be beneficial to investigate why no successful logins are reported in the logs. Is the SSH server configured correctly? Are there any
 issues with client-side authentication or connection setup?

Here is a sample bash script that can help detect and block malicious IP addresses:
```bash
#!/bin/bash

# Log file path
LOG_FILE="/var/log/auth.log"

# Threshold for number of failed login attempts within a certain time window (e.g., 5 attempts in 1 minute)
THRESHOLD=5
TIME_WINDOW=60

# IP address to block
BLOCKED_IP="192.168.1.100"

while read -r line; do
    if [[ $line =~ "login failed" ]]; then
        ip=$(echo "$line" | cut -d' ' -f2)
        attempts=$((attempts + 1))
        last_attempt_time=$(date -d "$(echo "$line" | cut -d' ' -f4)" +%s)

        if ((attempts > THRESHOLD)); then
            if (( $(($(date +%s) - $last_attempt_time)) < TIME_WINDOW )); then
                echo "Blocked IP: $ip"
                # Add code to block IP address using firewall rules or other means
            fi

            attempts=0
        fi
    fi
done < "$LOG_FILE"
```

My evaluation based on the LLM responses :

The output revealed general trends of unsuccessful attempts at the SSH-based logins, e.g. invalid usernames and wrong passwords, but it was too general and not detailed. It did not specify the particular timestamps, clusters of anomalies or sequences of unusual happening as made mandatory by the prompt. Moreover, the model made false assumptions on legitimate user behavior and provided remediation advice, which was beyond the scope of the task. The answer did not concentrate on anomalies detected on a timing basis and did not generate structured anomalies detection. In general, the deliverable was somewhat helpful and not as timely as required.
```

Reflection- Prompt Engineering :

In all three prompts, there was a certain trend that useful security prompts have to be specific, structured and close in scope. As the task became well delineated in response to the prompts, as identifying the brute-force IPs, grouping activity by source address, or detecting anomalies in the specific time windows, the model generated more structured and actionable results. Prompts with explicit instructions like the one distinguishing between invalid username attempts and unsuccessful password attempts or forcing severity rating to the user assisted the model in moving the processing to a deeper analysis level instead of merely superficial observations. Also, the presence of a structured format of output (e.g. sections per IP or clusters of anomalies) greatly enhanced the understandability and facilitated interpretation and comparison of the results.

The other significant observation was the effect of input size. The model tended to make loose summaries and overlook significant information when being used with the complete 2000 line of data. The same prompts were however, used with smaller chunks (e.g. 500 lines) and the outputs were more precise and there was a better identification of patterns, repeated behavior and timing-based anomalies. This demonstrates that timely performance is strongly associated not only with the wording but also the amount of information handled by the model during any specific time.

Conversely, weak and too general responses were obtained with vague or generic prompts, including the mere question of whether the log had suspicious activity on it. In such instances, the model was more likely to support general patterns of SSH attacks, but not to mention certain evidence in the logs. It also often did not adhere to restrictions e.g. not accepting remediation advice, unless these instructions were highlighted strongly.

All in all, the best prompt design decisions proved to be the definition of tasks and the scope through rigid restrictions, the presence of organized output specifications, and the instructions on how to work with the huge datasets. The least effective prompts were the open-ended prompts, but they were not directional, hence they gave generic and less helpful analysis.

References :

1. stratosphereips. (2025). *GitHub - stratosphereips/llm-log-analyzer: A small Python file that uses a local LLM, such as ollama, to analyze text files given a prompt. Designed for log files*. GitHub. https://github.com/stratosphereips/llm-log-analyzer

2. Kent, K., & Souppaya, M. (2006, September 13). *Guide to computer security log management*. CSRC. https://csrc.nist.gov/pubs/sp/800/92/final

3. ChatGPT, DeepSeek (Open AI Models).